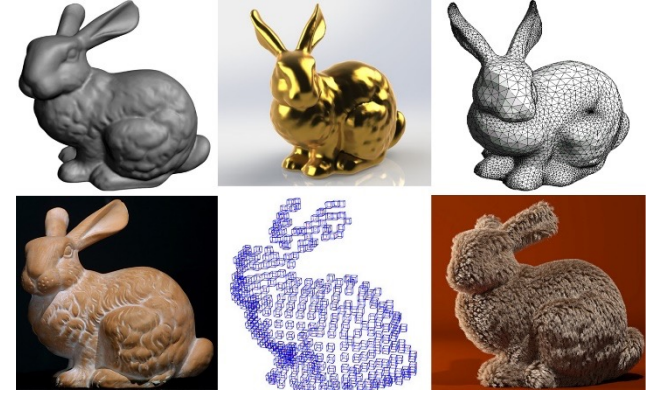




CS100433

Course Review

Junqiao Zhao 赵君峤

School of Computer Science and Technology

Tongji University

Well Done! Hardcore CGers!

Computer Graphics

- Part 1 Introduction
 - Basic Viewing and Modeling
 - Graphics Pipeline
 - Solid modeling
 - Modern OpenGL
 - GLSL

Computer Graphics

- Part 2 Viewing
 - 2D and 3D Transformation
 - Homogeneous coordinates
 - Affine transformations
 - 2D and 3D Viewing
 - Camera and Viewing frustum
 - Projection transformations and Clipping
 - Visible Surface Detection

Computer Graphics

- Part 3 Rendering
 - Rasterization
 - Light science
 - Light sources
 - Reflection model
 - Surface Shading
 - Texture mapping
 - Shadows

Computer Graphics

- Part 4 Adv. Rendering
 - Physically-based rendering
 - Ray-tracing
 - Neural rendering

Computer Graphics

- Part 5 Geometric modeling
 - Triangle mesh
 - Splines and subdivision

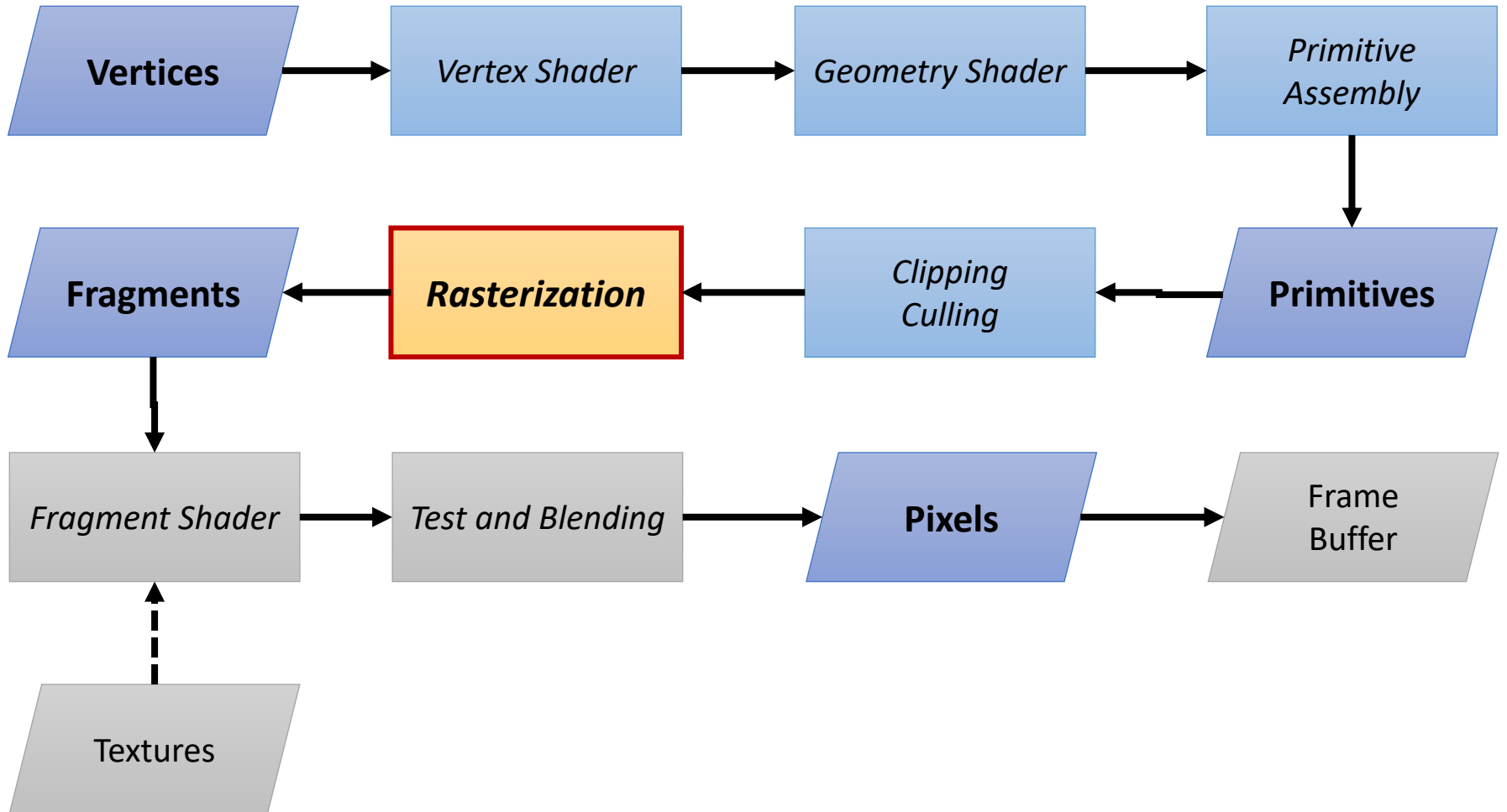
- Only shown you the **very basics** of Computer Graphics
- To further enrich your knowledge of CG, please attend CMU 15-462, Games101 open course
- Those who want to devote themselves to CG, please read
 - <https://s2024.siggraph.org/programs-events/>

Four themes

- Pipeline
- Viewing
- Rendering
- Geometry

Theme 1 Graphics Pipeline

- Can you draw it?
- What does programmable mean?
- Vertex vs Primitive vs Fragment?
 - Who poses more influence on the framerate of a CG application? Why?
- Compare it with Ray-tracing
 - Pros and Cons?

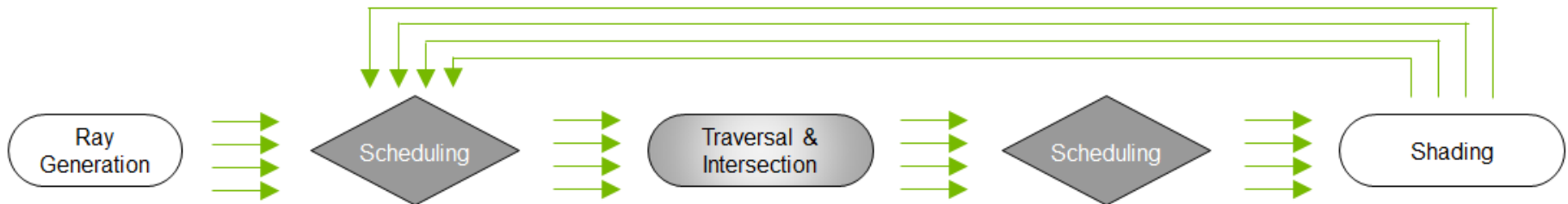


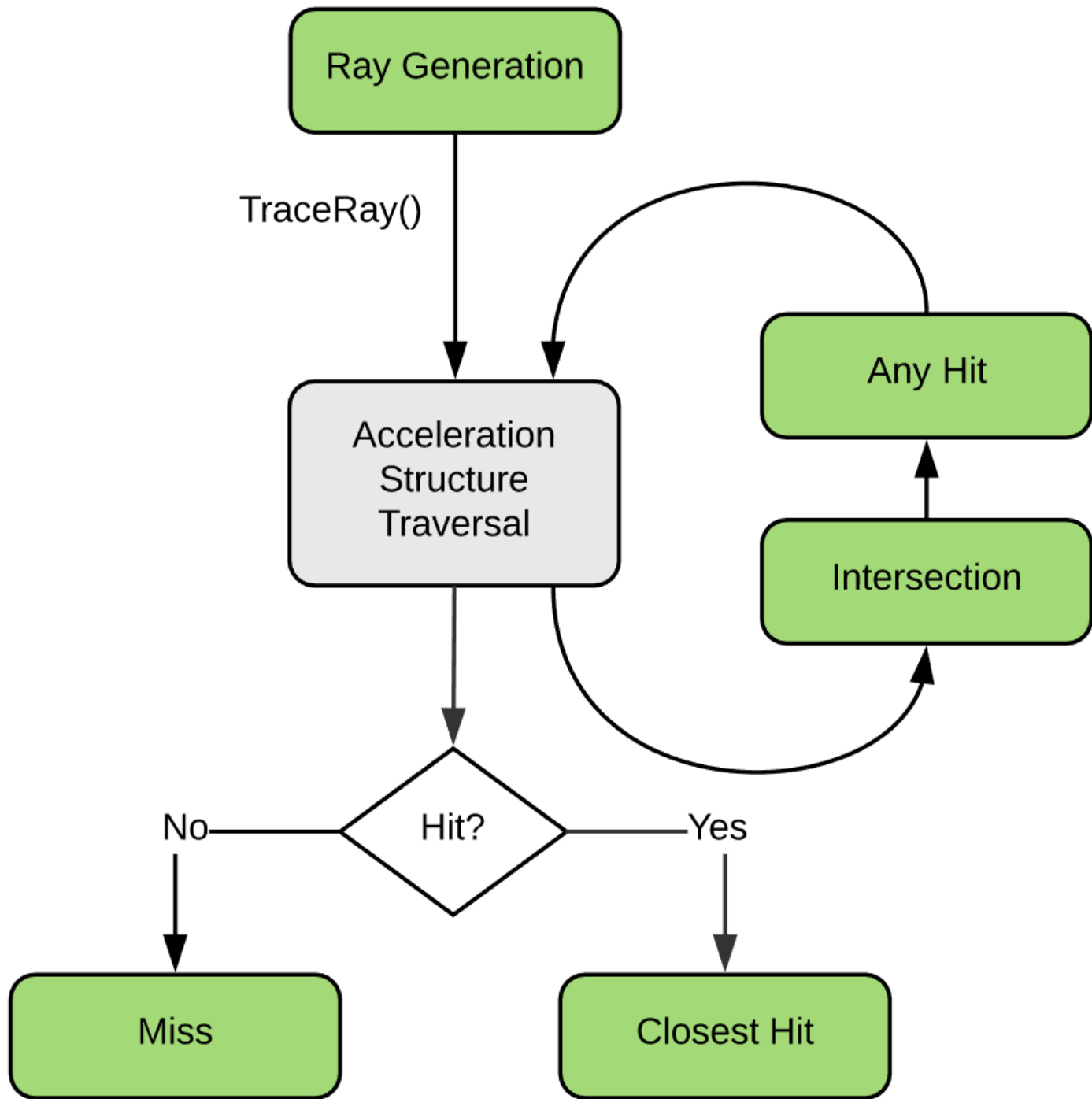
Raytracing Pipeline

Rasterization



Ray Tracing

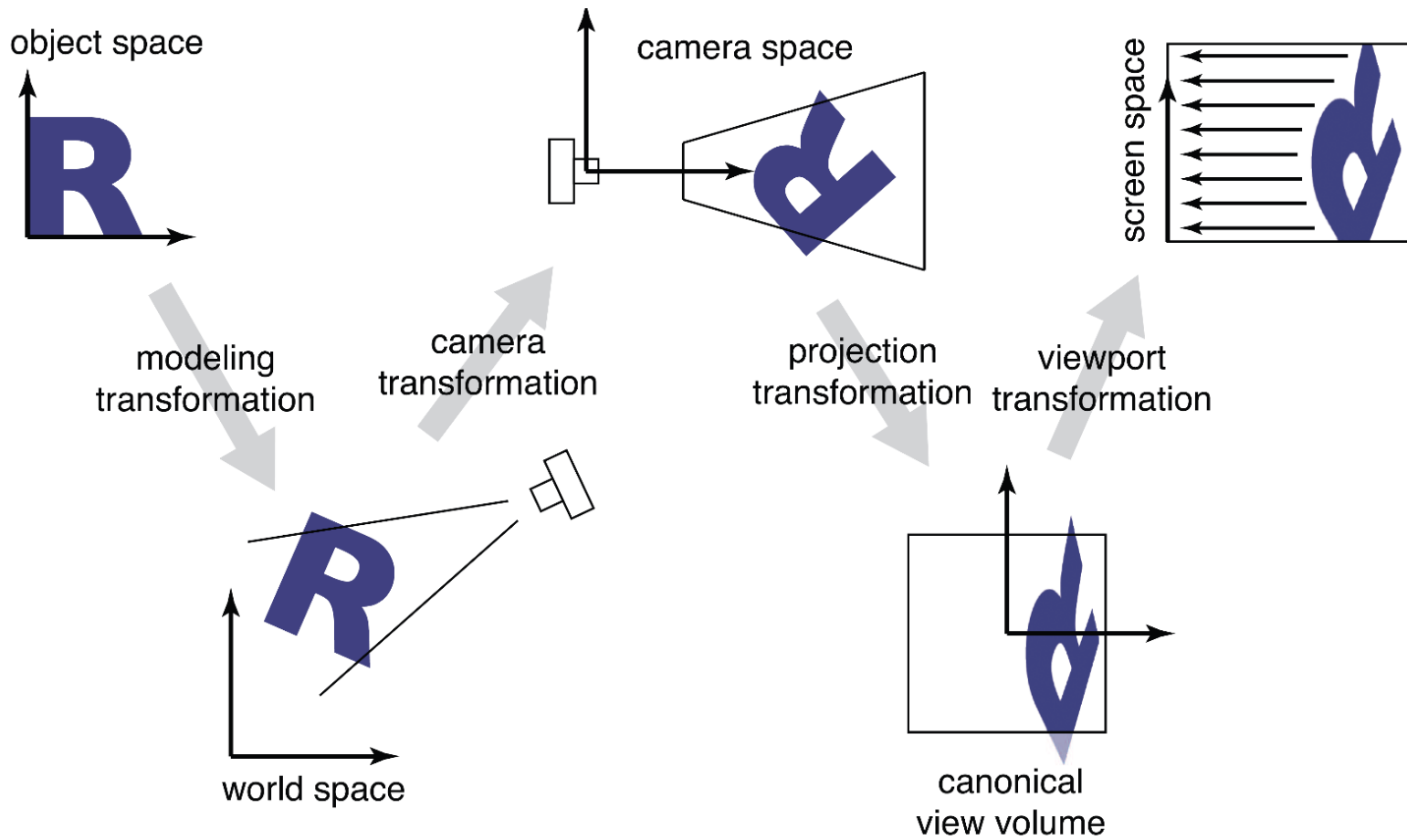


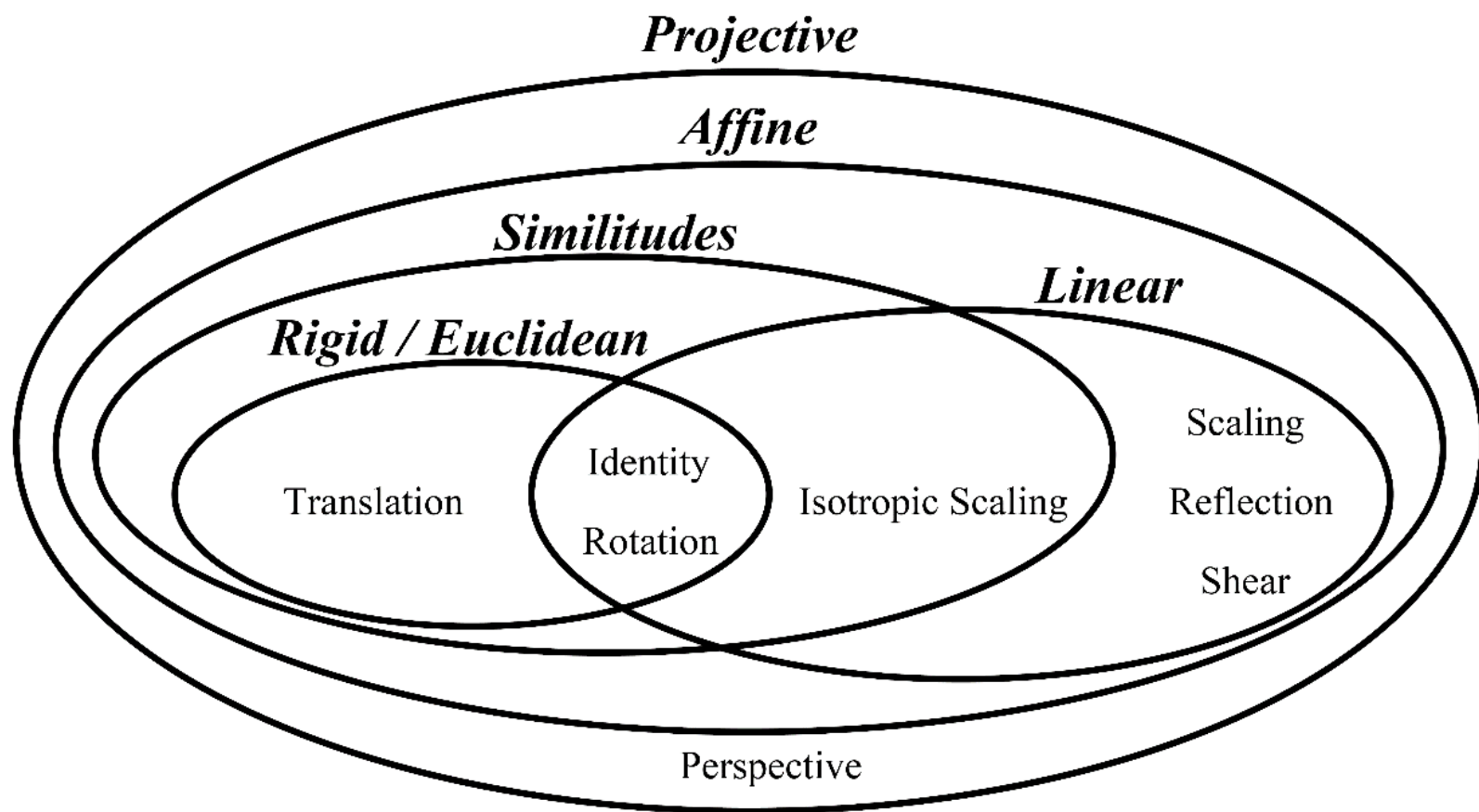


- Questions?

Theme 2 Viewing

- What coordinate spaces are involved in viewing?
- Which of the following transformations compose the Viewing Pipeline?
 - Affine transformation
 - Modeling transformation
 - Camera transformation
 - Rigid transformation
 - Viewport transformation
 - Linear transformation
 - Projection transformation
- In what order?
- Concatenation of transformations



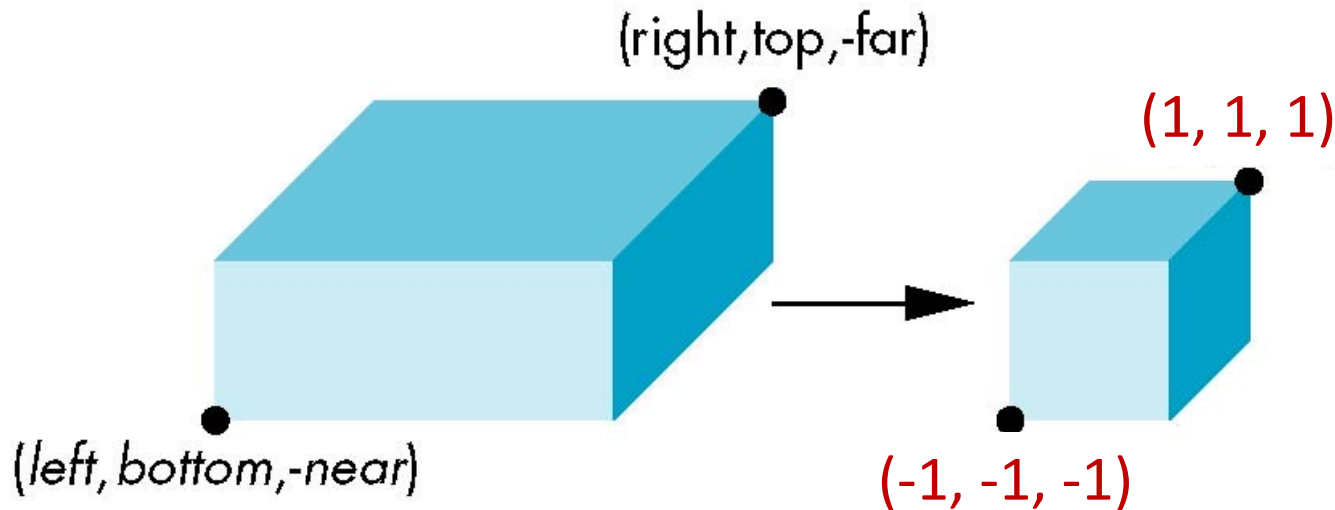


Theme 2 Viewing

- How to model a virtual camera?
 - pose
 - focal length
 - size of film
 - Which can be used for zooming?
- Orthogonal Projection and Perspective Projection
 - Viewing volume/frustum
 - Projection matrix
 - Normalization

Orthogonal Normalization

`glm::Ortho(left, right, bottom, top, near, far)`

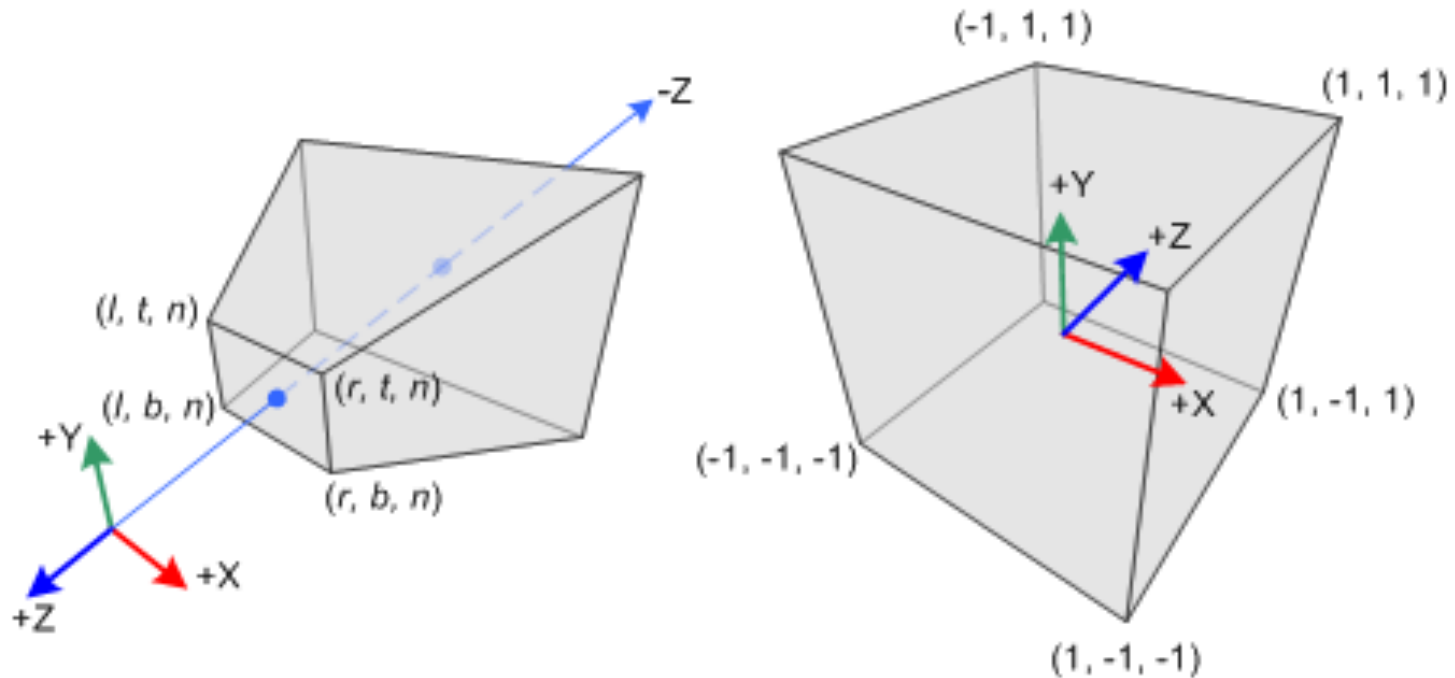


Orthogonal Matrix

- Two steps
 - Move center to origin
 $T(-(left+right)/2, -(bottom+top)/2, (near+far)/2))$
 - Scale to have sides of length 2
 $S(2/(right-left), 2/(top-bottom), 2/(-far -(- near)))$

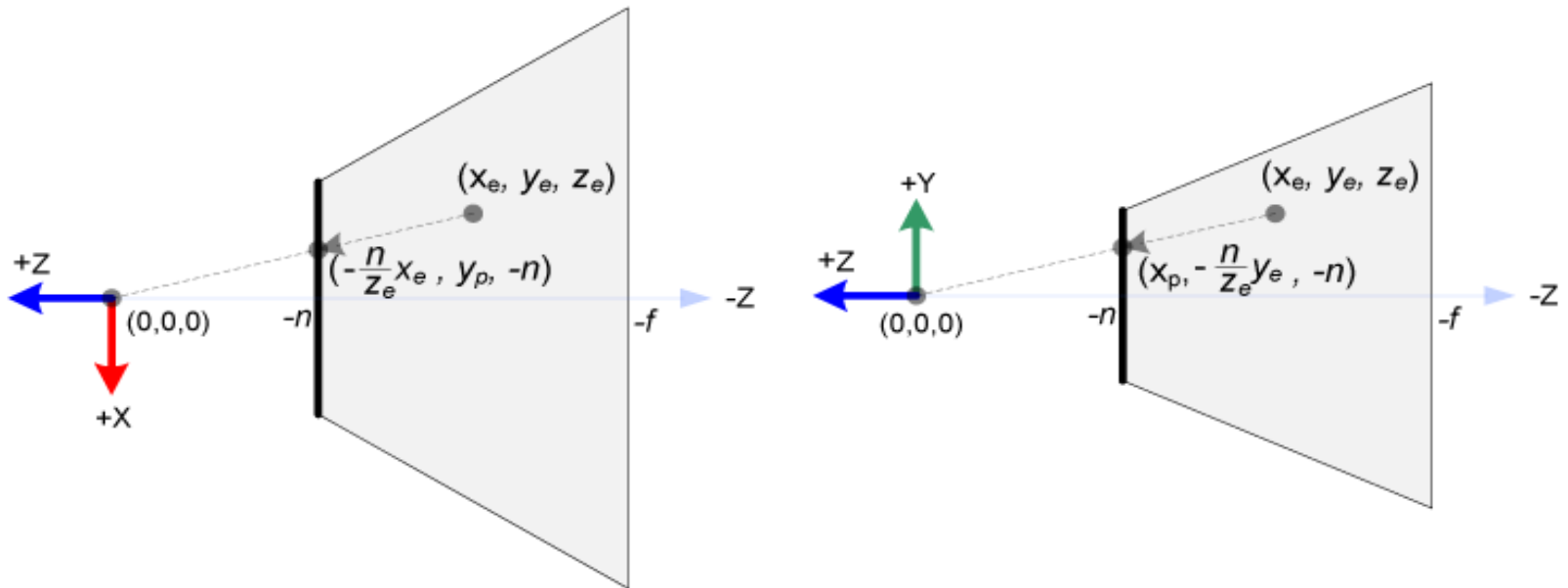
$$\mathbf{P} = \mathbf{ST} = \begin{bmatrix} \frac{2}{right - left} & 0 & 0 & -\frac{right + left}{right - left} \\ 0 & \frac{2}{top - bottom} & 0 & -\frac{top + bottom}{top - bottom} \\ 0 & 0 & \frac{2}{near - far} & -\frac{far + near}{far - near} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Normalize Perspective Transformation



Normalize Perspective Transformation

- $x_p = -\frac{n}{z_e} x_e$
- $y_p = -\frac{n}{z_e} y_e$



Normalize Perspective Transformation

- Normalize x_p and y_p instead
- Using the same “trick” as we did for orthogonal projection
- $x_n = \frac{2}{r-l} \cdot x_p - \frac{r+l}{r-l}$
- $y_n = \frac{2}{t-b} \cdot y_p - \frac{t+b}{t-b}$
- Substitute x_e and y_e into the equations
- $x_n = (\frac{2n}{r-l} \cdot x_e + \frac{r+l}{r-l} \cdot z_e) / -z_e$
- $y_n = (\frac{2n}{t-b} \cdot y_p + \frac{t+b}{t-b} \cdot z_e) / -z_e$

Normalize Perspective Transformation

$$\bullet \begin{pmatrix} x_c \\ y_c \\ z_c \\ w_c \end{pmatrix} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} x_e \\ y_e \\ z_e \\ 1 \end{pmatrix}$$

$$\mathbf{P} = \mathbf{M}_{\text{orth}} \mathbf{M}_{\text{pers}}$$

Theme 2 Viewing

- Why clipping in clip space rather than in NDC?
 - Clipping in Viewing frustum?
 - Clipping in NDC?
- Clipping algorithms
 - For lines
 - For polygons
- Painter's algorithm VS Z-buffering
 - Depth order

- Questions?

Theme 3 Rendering

- What 3D clues do you know until now?
- How many types of light sources do you know?
- Reflection model vs Shading method?
 - Why blinn-phong is better than phong?
 - Why phong shading is better than gourand shading?
- Material vs Texture?
- How to deal with multiple light sources simultaneously?
 - How dose the number of lights affect the performance?

Summary of Phong reflection

- Light components
 - Ambient I_a , Diffuse I_d and Specular I_s
- Material coefficients for each light component
 - k_a , k_d , k_s and α
- Therefore:
- $I = L_a k_a + L_d k_d \max(l \cdot n, 0) + L_s k_s \max(r \cdot v)^\alpha, 0)$

Summary of Phong reflection

- Attenuation

- $$I_{\text{atten}} = I_a + \frac{1}{a_0 + a_1 d + a_2 d^2} (I_d + I_s)$$

- How about If there are multiple lights?

- $$I = L_a k_a + \sum_{i=1}^m (L_{d,i} k_d \max(I \cdot n, 0) + L_{s,i} k_s \max((r_i \cdot v)^\alpha, 0))$$

- How about shadows?

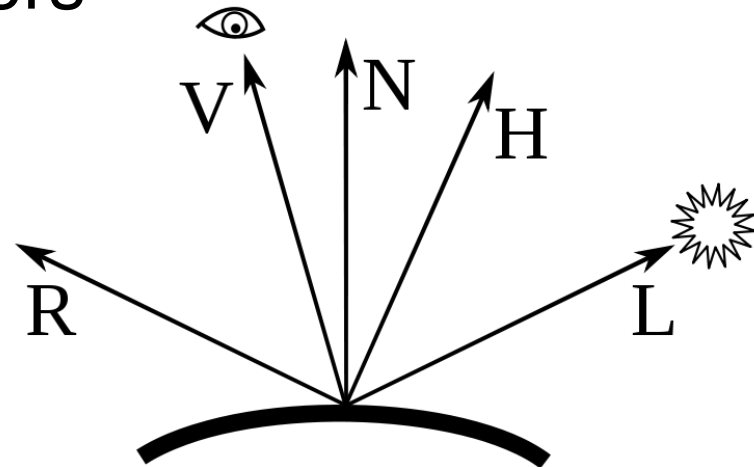
- $$I = L_a k_a + s_i \sum_{i=1}^m (L_{d,i} k_d \max(I \cdot n, 0) + L_{s,i} k_s \max((r_i \cdot v)^\alpha, 0))$$

Blinn-Phong reflection model

- A modification to the Phong reflection model developed by Jim Blinn
- In Phong model, one must continually recalculate the dot product of r and v . Instead, one calculates a halfway vector between the viewer and light-source vectors

$$\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|}$$

- $L_s k_s (r \cdot v)^\alpha \rightarrow L_s k_s (n \cdot h)^\alpha$



Theme 3 Rendering

- How to rasterize an arbitrary polygon?
- Mipmapping vs Bump-mapping vs Displacement mapping vs Cubic mapping?
- The flexible usages of texture objects?
- Shadow map vs Shadow volume?

Theme 4 Adv. Rendering

- What are the definition radiance and irradiance?
- What is the rendering equation?
- What is BRDF?

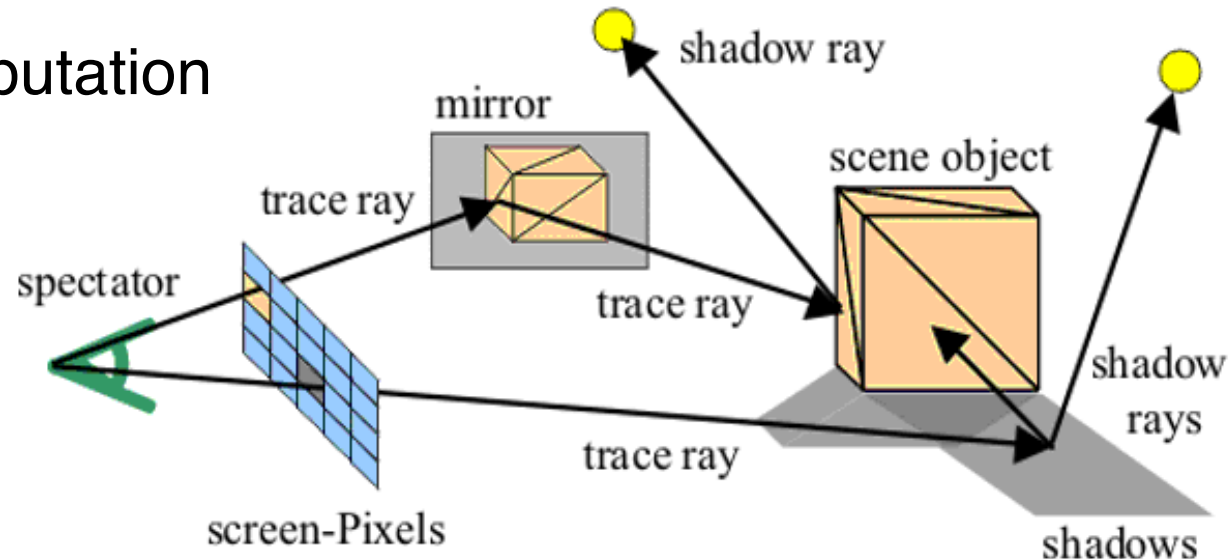
The diagram illustrates the rendering equation with various components labeled in red text and arrows:

- outgoing/observed radiance**: Points to $L_o(\mathbf{p}, \omega_o)$
- point of interest**: Points to $\mathbf{p}$
- direction of interest**: Points to ω_o
- emitted radiance (e.g., light source)**: Points to $L_e(\mathbf{p}, \omega_o)$
- all directions in hemisphere**: Points to the integration domain $\mathcal{H}^2$
- scattering function**: Points to $f_r(\mathbf{p}, \omega_i \rightarrow \omega_o)$
- incoming radiance**: Points to $L_i(\mathbf{p}, \omega_i)$ (circled in red)
- incoming direction**: Points to ω_i
- angle between incoming direction and normal**: Points to θ

$$L_o(\mathbf{p}, \omega_o) = L_e(\mathbf{p}, \omega_o) + \int_{\mathcal{H}^2} f_r(\mathbf{p}, \omega_i \rightarrow \omega_o) L_i(\mathbf{p}, \omega_i) \cos \theta d\omega_i$$

Theme 4 Ray-tracing

- What is a ray tree?
- How to render shadow in Ray tracing?
- How to render reflection in Ray tracing?
- How to accelerate Ray tracing?
 - BVH
 - Parallel Computation

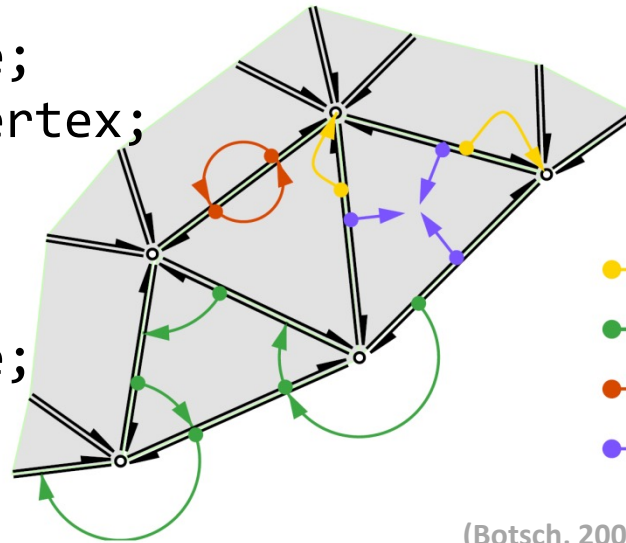


Theme 5 Geometry

- What do you think is a valid Geometry?
- What applications need mesh query?
- What is an appropriate representation to model
 - A Spherical ball?
 - A building?
 - A terrain?
 - An ocean?

Half-edge data structure

```
struct Halfedge {  
    Halfedge* next;  
    Halfedge* opposite;  
    Face* incident_face;  
    Vertex* incident_vertex;  
}  
  
struct Face{  
    Halfedge * halfedge;  
}  
  
struct Vertex{  
    Halfedge* incident_halfedge;  
}
```



- to_vertex
- next_halfedge
- opposite_halfedge
- face

(Botsch, 2007)

Curve

- GBT
- Spline
- Subdivision

- Questions?

Discussion

- How to generate 3D content using neural network?

Discussion

- How to modify a CG scene using neural network?

Discussion

- How to construct a dynamic world model?

Exam

- 90 minutes
- Q/A and Programming
- Answer in English as much as possible

- Questions?

- Thank you!

- cs1.tongji.edu.cn/course/CS100433